

ROYAUME DU MAROC
Ministere de l'Education Nationale

LYCEE MOULAY YOUSSEF - TANGER

BTS ELECTRONIQUE ET INFORMATIQUE INDUSTRIELLE

COURS RASPBERRY PI 4

Systeme Embarque sous Linux

Cours + Exemples + Exercices TD

PARTIE 1: Decouverte

- Presentation du RPi 4
- Architecture et composants
- Installation Raspberry Pi OS
- Configuration systeme
- Commandes Linux essentielles

PARTIE 2: Programmation

- GPIO avec Python
- Entrees/Sorties numeriques
- PWM et servomoteurs
- Communication I2C
- Communication SPI
- Communication serie UART

PARTIE 3: Applications

- Capteurs et actionneurs
- Interface homme-machine
- Serveur web embarque

PARTIE 4: Travaux Pratiques

- TP1 a TP5 progressifs
- Mini-projets
- Projets complets

Annee scolaire 2025-2026

1 Presentation du Raspberry Pi 4

1.1 Qu'est-ce que le Raspberry Pi ?

Le Raspberry Pi est un **ordinateur monocarte** (SBC - Single Board Computer) de la taille d'une carte de credit. Il a ete concu pour l'apprentissage de l'informatique et l'electronique.

Caracteristiques du Raspberry Pi 4 :

- Processeur ARM Cortex-A72 quad-core 1.5 GHz (64 bits)
- RAM : 2, 4 ou 8 Go LPDDR4
- 2 ports USB 3.0 + 2 ports USB 2.0
- 2 sorties micro-HDMI (4K)
- Port Gigabit Ethernet + WiFi 5 + Bluetooth 5.0
- 40 broches GPIO

1.2 Applications

Domaine	Exemples
Education	Apprentissage programmation, electronique
Domotique	Controle maison, automatisation
IoT	Capteurs connectes, datalogger
Robotique	Robots mobiles, bras articules
Media center	Kodi, Plex, streaming
Serveur	Web, fichiers, VPN, DNS

1.3 Comparaison avec microcontrolleurs

Caracteristique	Raspberry Pi 4	Arduino Uno	STM32F4
Type	Ordinateur	Microcontrolleur	Microcontrolleur
OS	Linux	Aucun	Aucun/RTOS
CPU	1.5 GHz quad-core	16 MHz	168 MHz
RAM	2-8 Go	2 Ko	192 Ko
GPIO	40 pins	20 pins	80+ pins
Prix	~45-80 EUR	~25 EUR	~15-30 EUR

2 Architecture et Composants

2.1 Schema de la carte

Le Raspberry Pi 4 Model B integre tous les composants necessaires sur une seule carte :

Composant	Description
SoC BCM2711	Processeur + GPU + controleurs
RAM LPDDR4	Memoire vive soudee
Slot microSD	Stockage systeme et donnees
USB-C	Alimentation 5V/3A
micro-HDMI x2	Sorties video 4K
Ethernet RJ45	Reseau Gigabit
GPIO 40 pins	Entrees/Sorties programmables
CSI	Camera (interface serie)
DSI	Ecran tactile officiel

2.2 Alimentation

Specifications d'alimentation :

- Tension : 5V DC (strictement)
- Courant : minimum 3A recommande
- Connecteur : USB Type-C
- Attention aux alimentations de mauvaise qualite (icone éclair)
- Alimentation officielle recommandee

2.3 Stockage

Type	Avantages	Inconvenients
microSD	Simple, economique	Lent, usure
SSD USB	Rapide, fiable	Plus cher
NVMe (HAT)	Tres rapide	Necessite HAT
Boot reseau	Centralise	Infrastructure requise

2.4 Accessoires essentiels

Accessoire	Obligatoire	Recommande
------------	-------------	------------

Alimentation 5V/3A	Oui	-
Carte microSD 16Go+	Oui	-
Boitier	-	Oui
Dissipateur/ventilateur	-	Oui
Cable HDMI	-	Pour debug
Clavier/souris	-	Pour config

3 Connecteur GPIO

3.1 Vue d'ensemble du GPIO

Le GPIO (General Purpose Input/Output) est un connecteur 40 broches permettant d'interfacer le Raspberry Pi avec des composants électroniques.

Caracteristiques GPIO :

- 26 broches GPIO programmables
- Niveaux logiques 3.3V (pas 5V tolerant !)
- Courant max par broche : 16 mA
- Courant total max GPIO : 50 mA
- Resistances pull-up/pull-down internes

3.2 Types de broches

Type	Broches	Description
3.3V	1, 17	Alimentation 3.3V (50mA max)
5V	2, 4	Alimentation 5V (depuis USB-C)
GND	6,9,14,20,25,30,34,39	Masse
GPIO	Voir tableau	Entrees/Sorties numeriques
I2C	3 (SDA), 5 (SCL)	Bus I2C
SPI	19,21,23,24,26	Bus SPI
UART	8 (TX), 10 (RX)	Port serie

3.3 Brochage GPIO (numeration BCM)

Pin	GPIO	Fonction	Fonction	GPIO	Pin
1	3.3V	-	-	5V	2
3	GPIO2	I2C SDA	-	5V	4
5	GPIO3	I2C SCL	GND	-	6
7	GPIO4	-	UART TX	GPIO14	8
9	GND	-	UART RX	GPIO15	10
11	GPIO17	-	-	GPIO18	12
13	GPIO27	-	GND	-	14
15	GPIO22	-	-	GPIO23	16

17	3.3V	-	-	GPIO24	18
19	GPIO10	SPI MOSI	GND	-	20
21	GPIO9	SPI MISO	-	GPIO25	22
23	GPIO11	SPI SCLK	-	GPIO8	24
25	GND	-	-	GPIO7	26

3.4 Brochage GPIO (suite)

Pin	GPIO	Fonction	Fonction	GPIO	Pin
27	ID_SD	EEPROM	EEPROM	ID_SC	28
29	GPIO5	-	GND	-	30
31	GPIO6	-	-	GPIO12	32
33	GPIO13	-	GND	-	34
35	GPIO19	-	-	GPIO16	36
37	GPIO26	-	-	GPIO20	38
39	GND	-	-	GPIO21	40

ATTENTION :

- Ne JAMAIS connecter du 5V sur une broche GPIO (destruction !)
- Ne JAMAIS dépasser 16 mA par broche
- Toujours utiliser des résistances de protection pour les LEDs
- Vérifier les connexions AVANT de mettre sous tension

3.5 Commande pinout

Commande pinout

```
# Afficher le brochage dans le terminal  
pinout
```

```
# Resultat : schema ASCII du GPIO avec  
# - Numerotation des broches  
# - Fonctions alternatives  
# - Informations sur la carte
```

4 Installation du Systeme

4.1 Raspberry Pi OS

Raspberry Pi OS (anciennement Raspbian) est le systeme d'exploitation officiel base sur Debian Linux.

Version	Desktop	Usage
Lite	Non	Serveur, headless, embarque
Desktop	Oui	Usage general
Desktop Full	Oui + apps	Debutants, education

4.2 Installation avec Raspberry Pi Imager

Etapas d'installation

1. Telecharger Raspberry Pi Imager depuis raspberrypi.com
2. Insérer la carte microSD dans le PC
3. Lancer Raspberry Pi Imager
4. Choisir l'OS : Raspberry Pi OS (32 ou 64 bits)
5. Choisir la carte SD
6. Cliquer sur la roue dentée pour configurer :
 - Nom d'hôte
 - SSH active
 - Utilisateur/mot de passe
 - WiFi (SSID et mot de passe)
 - Fuseau horaire
7. Cliquer sur ECRIRE
8. Insérer la SD dans le Raspberry Pi

4.3 Premier demarrage

Mode graphique :

- Connecter écran, clavier, souris
- Suivre l'assistant de configuration

Mode headless (sans écran) :

- Se connecter en SSH : `ssh pi@raspberrypi.local`
- Mot de passe configure dans Imager

4.4 Configuration avec raspi-config

raspi-config

```
# Lancer l'outil de configuration
sudo raspi-config

# Options importantes :
# 1 System Options - Mot de passe, hostname, boot
# 2 Display Options - Resolution, overscan
# 3 Interface Options - SSH, VNC, SPI, I2C, Serial
# 4 Performance - Overclock, GPU memory
# 5 Localisation - Langue, timezone, clavier
# 6 Advanced - Expand filesystem
```

4.5 Activer les interfaces

Interface	Menu raspi-config	Usage
SSH	Interface Options > SSH	Acces distant securise
VNC	Interface Options > VNC	Bureau a distance
SPI	Interface Options > SPI	Bus SPI
I2C	Interface Options > I2C	Bus I2C
Serial	Interface Options > Serial	UART
Camera	Interface Options > Camera	Module camera

4.6 Mise a jour du systeme

Mise a jour systeme

```
# Mettre a jour la liste des paquets
sudo apt update

# Mettre a jour les paquets installes
sudo apt upgrade -y

# Mettre a jour le firmware (optionnel)
sudo rpi-update

# Redemarrer si necessaire
sudo reboot
```

5 Commandes Linux Essentielles

5.1 Navigation et fichiers

Commande	Description	Exemple
pwd	Repertoire courant	pwd
ls	Lister fichiers	ls -la
cd	Changer repertoire	cd /home/pi
mkdir	Creer repertoire	mkdir projet
rm	Supprimer	rm fichier.txt
cp	Copier	cp src dest
mv	Deplacer/renommer	mv ancien nouveau
cat	Afficher contenu	cat fichier.txt
nano	Editeur texte	nano script.py

5.2 Permissions

Permissions

```
# Voir les permissions
ls -l
# -rwxr-xr-x 1 pi pi 1234 date fichier.py
# r=read, w=write, x=execute
# [user][group][others]

# Modifier permissions
chmod +x script.py # Rendre executable
chmod 755 script.py # rwxr-xr-x
chmod 644 fichier.txt # rw-r--r--

# Changer proprietaire
sudo chown pi:pi fichier.txt
```

5.3 Gestion des processus

Commande	Description
ps aux	Lister tous les processus
top / htop	Moniteur systeme interactif
kill PID	Terminer un processus

killall nom	Terminer par nom
Ctrl+C	Interrompre programme
Ctrl+Z	Suspendre programme
bg / fg	Arriere-plan / premier plan

5.4 Reseau

Commandes reseau

```
# Adresse IP
hostname -I
ip addr show

# Test connexion
ping google.com

# Configuration WiFi
sudo nano /etc/wpa_supplicant/wpa_supplicant.conf

# Scanner WiFi
sudo iwlist wlan0 scan | grep ESSID

# Connexion SSH
ssh pi@192.168.1.100
```

5.5 Systeme

Commande	Description
sudo reboot	Redemarrer
sudo shutdown -h now	Eteindre
df -h	Espace disque
free -h	Memoire RAM
vcgencmd measure_temp	Temperature CPU
lsusb	Peripheriques USB
lsblk	Disques et partitions
dmesg	Messages systeme

5.6 Installation de logiciels

Gestion des paquets

```
# Rechercher un paquet
apt search nom_paquet

# Installer un paquet
sudo apt install nom_paquet

# Desinstaller
sudo apt remove nom_paquet

# Installer pip (Python)
sudo apt install python3-pip

# Installer bibliotheque Python
pip3 install nom_bibliotheque
```

6 GPIO avec Python

6.1 Bibliotheque RPi.GPIO

RPi.GPIO est la bibliotheque standard pour controler le GPIO en Python.

Configuration de base

```
import RPi.GPIO as GPIO
import time

# Mode de numerotation
GPIO.setmode(GPIO.BCM) # Numerotation GPIO (recommande)
# ou
GPIO.setmode(GPIO.BOARD) # Numerotation physique des pins

# Desactiver les warnings
GPIO.setwarnings(False)
```

6.2 Configuration des broches

Configuration des broches

```
# Configurer en sortie
GPIO.setup(17, GPIO.OUT)

# Configurer en entree
GPIO.setup(18, GPIO.IN)

# Entree avec pull-up interne
GPIO.setup(18, GPIO.IN, pull_up_down=GPIO.PUD_UP)

# Entree avec pull-down interne
GPIO.setup(18, GPIO.IN, pull_up_down=GPIO.PUD_DOWN)

# Configuration multiple
GPIO.setup([17, 27, 22], GPIO.OUT)
```

6.3 Ecriture et lecture

Lecture et ecriture

```
# Ecrire sur une sortie
GPIO.output(17, GPIO.HIGH) # ou True ou 1
GPIO.output(17, GPIO.LOW) # ou False ou 0

# Lire une entree
etat = GPIO.input(18)
if etat == GPIO.HIGH:
    print("Niveau haut")

# Nettoyage (IMPORTANT a la fin)
GPIO.cleanup()
```

6.4 Exemple : Clignotement LED

led_blink.py

```
import RPi.GPIO as GPIO
import time

LED_PIN = 17

GPIO.setmode(GPIO.BCM)
GPIO.setup(LED_PIN, GPIO.OUT)

try:
    while True:
        GPIO.output(LED_PIN, GPIO.HIGH)
        print("LED ON")
        time.sleep(0.5)

        GPIO.output(LED_PIN, GPIO.LOW)
        print("LED OFF")
        time.sleep(0.5)

except KeyboardInterrupt:
    print("Arret")

finally:
    GPIO.cleanup()
```

Schema de cablage LED :

GPIO17 ---[Resistance 330 ohms]---[LED]--- GND

La resistance limite le courant a environ 10 mA :

$$I = (3.3V - 2V) / 330 = 4 \text{ mA (securitaire)}$$

6.5 Exemple : Lecture bouton

button_led.py

```
import RPi.GPIO as GPIO
import time

BUTTON_PIN = 18
LED_PIN = 17

GPIO.setmode(GPIO.BCM)
GPIO.setup(LED_PIN, GPIO.OUT)
GPIO.setup(BUTTON_PIN, GPIO.IN, pull_up_down=GPIO.PUD_UP)

try:
    while True:
        if GPIO.input(BUTTON_PIN) == GPIO.LOW: # Appuye
            GPIO.output(LED_PIN, GPIO.HIGH)
            print("Bouton appuye")
        else:
            GPIO.output(LED_PIN, GPIO.LOW)
            time.sleep(0.1) # Anti-rebond simple

except KeyboardInterrupt:
    pass
finally:
    GPIO.cleanup()
```

6.6 Interruptions

Interruptions GPIO

```
# Detection d'evenement avec callback
def button_callback(channel):
    print(f"Bouton detecte sur GPIO {channel}")

GPIO.setup(18, GPIO.IN, pull_up_down=GPIO.PUD_UP)

# Ajouter detection d'evenement
GPIO.add_event_detect(18, GPIO.FALLING,
    callback=button_callback,
    bouncetime=200) # Anti-rebond 200ms

# Attendre
try:
    while True:
        time.sleep(1)
except KeyboardInterrupt:
    GPIO.cleanup()
```

7 PWM et Servomoteurs

7.1 PWM (Pulse Width Modulation)

Le PWM permet de controler la puissance moyenne en variant le rapport cyclique d'un signal carre.

Applications du PWM :

- Variation de luminosite des LEDs
- Controle de vitesse des moteurs DC
- Positionnement des servomoteurs
- Generation de signaux audio

7.2 PWM logiciel

led_pwm.py - Variation de luminosite

```
import RPi.GPIO as GPIO
import time

LED_PIN = 18

GPIO.setmode(GPIO.BCM)
GPIO.setup(LED_PIN, GPIO.OUT)

# Creer objet PWM (pin, frequence Hz)
pwm = GPIO.PWM(LED_PIN, 1000) # 1 kHz

# Demarrer avec rapport cyclique 0%
pwm.start(0)

try:
    # Augmenter progressivement la luminosite
    for dc in range(0, 101, 5):
        pwm.ChangeDutyCycle(dc) # 0-100%
        print(f"Duty cycle: {dc}%")
        time.sleep(0.1)

    # Diminuer
    for dc in range(100, -1, -5):
        pwm.ChangeDutyCycle(dc)
        time.sleep(0.1)

finally:
    pwm.stop()
    GPIO.cleanup()
```

7.3 Controle de servomoteur

Les servomoteurs utilisent un signal PWM de 50 Hz avec un rapport cyclique de 2.5% a 12.5% pour les positions 0 a 180 degrees.

Angle	Rapport cyclique	Impulsion
0 deg	2.5%	0.5 ms
90 deg	7.5%	1.5 ms
180 deg	12.5%	2.5 ms

servo_control.py

```
import RPi.GPIO as GPIO
import time

SERVO_PIN = 18

GPIO.setmode(GPIO.BCM)
GPIO.setup(SERVO_PIN, GPIO.OUT)

# PWM a 50 Hz pour servomoteur
pwm = GPIO.PWM(SERVO_PIN, 50)
pwm.start(0)

def set_angle(angle):
    """Convertir angle (0-180) en rapport cyclique"""
    duty = 2.5 + (angle / 180.0) * 10
    pwm.ChangeDutyCycle(duty)
    time.sleep(0.3)
    pwm.ChangeDutyCycle(0) # Eviter vibrations

try:
    set_angle(0) # Position 0 deg
    time.sleep(1)
    set_angle(90) # Position 90 deg
    time.sleep(1)
    set_angle(180) # Position 180 deg

finally:
    pwm.stop()
    GPIO.cleanup()
```

Cablage servomoteur :

- Fil rouge : 5V (alimentation externe recommandee)
- Fil noir/marron : GND
- Fil orange/jaune : Signal (GPIO)

8 Communication I2C

8.1 Presentation du bus I2C

I2C (Inter-Integrated Circuit) est un bus serie synchrone permettant de connecter plusieurs peripheriques avec seulement 2 fils.

Caracteristiques I2C :

- 2 fils : SDA (donnees) et SCL (horloge)
- Communication maitre-esclave
- Adressage sur 7 bits (jusqu'a 127 peripheriques)
- Vitesses : 100 kHz (standard), 400 kHz (fast)
- Broches RPi : GPIO2 (SDA), GPIO3 (SCL)

8.2 Activation et detection

Detection I2C

```
# Activer I2C via raspi-config
sudo raspi-config
# Interface Options > I2C > Enable

# Installer les outils I2C
sudo apt install i2c-tools

# Detecter les peripheriques I2C
sudo i2cdetect -y 1

# Resultat : grille avec adresses des peripheriques
# 0 1 2 3 4 5 6 7 8 9 a b c d e f
# 20: -- -- -- -- -- -- -- 27 -- -- -- -- -- -- --
# 40: -- -- -- -- -- -- -- 48 -- -- -- -- -- -- --
# Le 27 = LCD, 48 = capteur temperature
```

8.3 Peripheriques I2C courants

Peripherique	Adresse	Usage
PCF8574	0x20-0x27	Expandeur GPIO 8 bits
LCD 16x2	0x27	Afficheur LCD
BMP280	0x76/0x77	Pression, temperature
ADS1115	0x48	ADC 16 bits
MPU6050	0x68	Accelerometre, gyroscope
DS3231	0x68	Horloge temps reel

8.4 Exemple : Capteur BME280

bme280_read.py

```
# Installation bibliotheque
# pip3 install adafruit-circuitpython-bme280

import board
import adafruit_bme280

# Initialisation I2C
i2c = board.I2C()
bme280 = adafruit_bme280.Adafruit_BME280_I2C(i2c, address=0x76)

# Lecture
print(f"Temperature : {bme280.temperature:.1f} C")
print(f"Humidite : {bme280.humidity:.1f} %")
print(f"Pression : {bme280.pressure:.1f} hPa")
```

8.5 Exemple : LCD I2C 16x2

lcd_i2c.py

```
# Installation : pip3 install RPLCD
from RPLCD.i2c import CharLCD
import time

# Initialisation LCD (adresse 0x27)
lcd = CharLCD(i2c_expander='PCF8574',
address=0x27,
cols=16, rows=2)

# Effacer et ecrire
lcd.clear()
lcd.write_string('Hello RPi!')

# Deuxieme ligne
lcd.cursor_pos = (1, 0)
lcd.write_string('BTS Tanger')

time.sleep(5)
lcd.close()
```

9 Communication SPI

9.1 Presentation du bus SPI

SPI (Serial Peripheral Interface) est un bus serie synchrone full-duplex, plus rapide que I2C mais necessitant plus de fils.

Caracteristiques SPI :

- 4 fils : MOSI, MISO, SCLK, CS
- Full-duplex (envoi et reception simultanes)
- Pas d'adressage (selection par CS)
- Vitesse jusqu'a plusieurs MHz
- Broches RPi : GPIO10 (MOSI), GPIO9 (MISO), GPIO11 (SCLK)

9.2 Broches SPI

Signal	GPIO	Pin	Description
MOSI	GPIO10	19	Master Out Slave In
MISO	GPIO9	21	Master In Slave Out
SCLK	GPIO11	23	Horloge
CE0	GPIO8	24	Chip Enable 0
CE1	GPIO7	26	Chip Enable 1

9.3 Activation SPI

Activation SPI

```
# Activer SPI via raspi-config
sudo raspi-config
# Interface Options > SPI > Enable

# Verifier l'activation
ls /dev/spi*
# Doit afficher : /dev/spidev0.0 /dev/spidev0.1

# Installer spidev pour Python
pip3 install spidev
```

9.4 Exemple : ADC MCP3008

Le MCP3008 est un convertisseur analogique-numerique 10 bits avec 8 canaux.

mcp3008_read.py

```
import spidev
import time

spi = spidev.SpiDev()
spi.open(0, 0) # Bus 0, Device 0 (CE0)
spi.max_speed_hz = 1000000 # 1 MHz

def read_adc(channel):
    """Lire canal ADC 0-7"""
    if channel < 0 or channel > 7:
        return -1
    # Commande MCP3008
    cmd = [1, (8 + channel) << 4, 0]
    response = spi.xfer2(cmd)
    # Extraire valeur 10 bits
    value = ((response[1] & 3) << 8) + response[2]
    return value

try:
    while True:
        val = read_adc(0) # Canal 0
        voltage = val * 3.3 / 1023
        print(f"ADC: {val} = {voltage:.2f}V")
        time.sleep(0.5)
finally:
    spi.close()
```

10 Communication Serie UART

10.1 Presentation UART

UART (Universal Asynchronous Receiver/Transmitter) permet la communication serie asynchrone point a point.

Caracteristiques UART :

- 2 fils : TX (emission) et RX (reception)
- Asynchrone (pas d'horloge)
- Vitesses standards : 9600, 115200 bauds
- Format : 8N1 (8 bits, pas de parite, 1 stop bit)
- Broches RPi : GPIO14 (TX), GPIO15 (RX)

10.2 Configuration UART

Configuration UART

```
# Activer UART via raspi-config
sudo raspi-config
# Interface Options > Serial Port
# - Login shell over serial: No
# - Serial port hardware enabled: Yes

# Verifier le port serie
ls -l /dev/serial*
# /dev/serial0 -> ttyS0 (mini UART)
# /dev/serial1 -> ttyAMA0 (PL011 UART)

# Installer pyserial
pip3 install pyserial
```

10.3 Exemple : Communication serie

uart_comm.py

```
import serial
import time

# Ouvrir port serie
ser = serial.Serial(
    port='/dev/serial0',
    baudrate=9600,
    timeout=1
)

# Envoi de donnees
ser.write(b'Hello from RPi!\n')

# Reception de donnees
while True:
    if ser.in_waiting > 0:
        data = ser.readline().decode('utf-8').strip()
        print(f"Recu: {data}")
        time.sleep(0.1)

ser.close()
```

10.4 Communication avec Arduino

Cablage RPi - Arduino :

- RPi TX (GPIO14) --- Arduino RX
- RPi RX (GPIO15) --- Arduino TX
- RPi GND --- Arduino GND

ATTENTION : Le RPi utilise du 3.3V et l'Arduino du 5V.
Utiliser un diviseur de tension sur Arduino TX -> RPi RX

10.5 Exemple : Lecture capteur GPS

gps_read.py

```
import serial

# Module GPS sur UART
gps = serial.Serial('/dev/serial0', 9600, timeout=1)

def parse_nmea(sentence):
    """Parser trame NMEA basique"""
    if sentence.startswith('$GPGGA'):
        parts = sentence.split(',')
        if len(parts) >= 6:
            lat = parts[2]
            lon = parts[4]
            return lat, lon
        return None, None

while True:
    line = gps.readline().decode('utf-8', errors='ignore')
    if line:
        lat, lon = parse_nmea(line)
        if lat and lon:
            print(f"Position: {lat}, {lon}")
```

11 Capteurs et Actionneurs

11.1 Capteur de temperature DS18B20

ds18b20.py

```
# Activer 1-Wire dans /boot/config.txt
# dtoverlay=w1-gpio

import os
import time

# Charger modules
os.system('modprobe w1-gpio')
os.system('modprobe w1-therm')

# Trouver le capteur
base_dir = '/sys/bus/w1/devices/'
device_folder = [f for f in os.listdir(base_dir)
if f.startswith('28-')][0]
device_file = base_dir + device_folder + '/w1_slave'

def read_temp():
    with open(device_file, 'r') as f:
        lines = f.readlines()
        equals_pos = lines[1].find('t=')
        temp_c = float(lines[1][equals_pos+2:]) / 1000.0
    return temp_c

print(f"Temperature: {read_temp():.1f} C")
```

11.2 Capteur ultrason HC-SR04

hcsr04.py

```
import RPi.GPIO as GPIO
import time

TRIG = 23
ECHO = 24

GPIO.setmode(GPIO.BCM)
GPIO.setup(TRIG, GPIO.OUT)
GPIO.setup(ECHO, GPIO.IN)

def mesure_distance():
    # Pulse trigger
    GPIO.output(TRIG, True)
    time.sleep(0.00001)
    GPIO.output(TRIG, False)

    # Mesurer echo
    while GPIO.input(ECHO) == 0:
        debut = time.time()
    while GPIO.input(ECHO) == 1:
        fin = time.time()

    # Calculer distance
    duree = fin - debut
    distance = duree * 34300 / 2 # cm
    return distance

print(f"Distance: {mesure_distance():.1f} cm")
```

11.3 Capteur DHT11/DHT22

dht_read.py

```
# Installation : pip3 install adafruit-circuitpython-dht
# sudo apt install libgpiod2

import adafruit_dht
import board
import time

# DHT11 sur GPIO4
dht = adafruit_dht.DHT11(board.D4)
# Pour DHT22 : dht = adafruit_dht.DHT22(board.D4)

while True:
    try:
        temp = dht.temperature
        hum = dht.humidity
        print(f"Temp: {temp}C Humidite: {hum}%")
    except RuntimeError as e:
        print(f"Erreur: {e}")
        time.sleep(2)
```

11.4 Controle moteur DC avec L298N

motor_l298n.py

```
import RPi.GPIO as GPIO
import time

# Pins L298N
ENA = 18 # PWM vitesse
IN1 = 23 # Direction
IN2 = 24

GPIO.setmode(GPIO.BCM)
GPIO.setup([ENA, IN1, IN2], GPIO.OUT)

pwm = GPIO.PWM(ENA, 1000)
pwm.start(0)

def forward(speed):
    GPIO.output(IN1, GPIO.HIGH)
    GPIO.output(IN2, GPIO.LOW)
    pwm.ChangeDutyCycle(speed)

def backward(speed):
    GPIO.output(IN1, GPIO.LOW)
    GPIO.output(IN2, GPIO.HIGH)
    pwm.ChangeDutyCycle(speed)

def stop():
    pwm.ChangeDutyCycle(0)

forward(50) # 50% vitesse
time.sleep(2)
stop()
```

12 Serveur Web Embarque

12.1 Flask - Serveur web léger

Flask permet de créer facilement un serveur web pour contrôler le GPIO depuis un navigateur.

flask_basic.py

```
# Installation : pip3 install flask

from flask import Flask

app = Flask(__name__)

@app.route('/')
def index():
    return 'Hello from Raspberry Pi!'

@app.route('/led/')
def led(state):
    if state == 'on':
        return 'LED allumee'
    else:
        return 'LED eteinte'

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=True)
```

12.2 Contrôle GPIO via Web

flask_gpio.py

```
from flask import Flask, render_template_string
import RPi.GPIO as GPIO

app = Flask(__name__)

LED_PIN = 17
GPIO.setmode(GPIO.BCM)
GPIO.setup(LED_PIN, GPIO.OUT)

HTML = '''
Controle LED
ON
OFF
Etat: {{ etat }}
'''

@app.route('/')
def index():
    etat = GPIO.input(LED_PIN)
    return render_template_string(HTML, etat='ON' if etat else 'OFF')

@app.route('/led/')
def led(state):
    GPIO.output(LED_PIN, state == 'on')
    return index()

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```


12.3 API REST pour capteurs

api_sensors.py

```
from flask import Flask, jsonify
import random # Simuler capteur

app = Flask(__name__)

@app.route('/api/temperature')
def get_temp():
    # Remplacer par vraie lecture capteur
    temp = random.uniform(20, 30)
    return jsonify({
        'temperature': round(temp, 1),
        'unite': 'C'
    })

@app.route('/api/humidity')
def get_humidity():
    hum = random.uniform(40, 80)
    return jsonify({
        'humidity': round(hum, 1),
        'unite': '%'
    })

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Acces au serveur :

- Depuis le RPi : `http://localhost:5000`
- Depuis le reseau : `http://[IP_RASPBERRY]:5000`
- Trouver l'IP : `hostname -I`

TRAVAUX PRATIQUES

TP1 : Prise en main

Exercice 1 : Installer Raspberry Pi OS sur une carte SD et configurer le WiFi et SSH.

Exercice 2 : Se connecter en SSH et executer les commandes de base (ls, cd, pwd, nano).

Exercice 3 : Afficher la temperature du CPU avec vcgencmd et creer un script qui l'affiche toutes les 5 secondes.

Exercice 4 : Ecrire un script Python qui affiche 'Hello BTS' et l'executer.

TP2 : GPIO - LEDs et boutons

Exercice 5 : Faire clignoter une LED a 1 Hz pendant 10 secondes.

Exercice 6 : Realiser un feu tricolore (3 LEDs) avec sequence : Vert 3s, Jaune 1s, Rouge 3s.

Exercice 7 : Commander une LED avec un bouton poussoir.

Exercice 8 : Compteur binaire 4 bits sur 4 LEDs avec bouton d'incrementation.

TP3 : PWM et servomoteurs

Exercice 9 : Variation progressive de luminosite d'une LED (fade in/out).

Exercice 10 : Commander un servomoteur avec 3 positions : 0, 90, 180 degres via boutons.

Exercice 11 : Controler la vitesse d'un moteur DC avec potentiometre (MCP3008 + L298N).

TP4 : Capteurs

Exercice 12 : Lire la temperature avec DS18B20 et l'afficher toutes les secondes.

Exercice 13 : Mesurer une distance avec HC-SR04 et allumer une LED si distance < 20 cm.

Exercice 14 : Afficher temperature et humidite (DHT11) sur LCD I2C.

Exercice 15 : Datalogger : enregistrer temperature dans un fichier CSV toutes les minutes.

TP5 : Projets

Projet 1 : Station meteo avec capteurs + affichage LCD + enregistrement.

Projet 2 : Systeme d'alarme avec detecteur de mouvement PIR et buzzer.

Projet 3 : Robot suiveur de ligne avec 2 moteurs DC et capteurs IR.

Projet 4 : Interface web pour controler plusieurs LEDs et lire capteurs.

Projet 5 : Systeme domotique : controle lumiere + temperature + volets.

Solutions et indices

Solution Exercice 6 : Feu tricolore

```
import RPi.GPIO as GPIO
import time

VERT, JAUNE, ROUGE = 17, 27, 22
GPIO.setmode(GPIO.BCM)
GPIO.setup([VERT, JAUNE, ROUGE], GPIO.OUT)

try:
    while True:
        GPIO.output(VERT, True)
        time.sleep(3)
        GPIO.output(VERT, False)
        GPIO.output(JAUNE, True)
        time.sleep(1)
        GPIO.output(JAUNE, False)
        GPIO.output(ROUGE, True)
        time.sleep(3)
        GPIO.output(ROUGE, False)
finally:
    GPIO.cleanup()
```

Indices pour les autres exercices :

Ex 8 : Utiliser une variable compteur et bin(compteur) pour convertir
Ex 11 : Lire ADC 0-1023, convertir en 0-100 pour PWM
Ex 13 : Ajouter condition if distance < 20 apres mesure
Ex 15 : Utiliser datetime et csv.writer
Projet 3 : Logique : si gauche=noir tourner gauche, si droite=noir tourner droite

RECAPITULATIF

Commandes essentielles

Action	Commande
Connexion SSH	ssh pi@raspberrypi.local
Configuration	sudo raspi-config
Mise a jour	sudo apt update && sudo apt upgrade
Temperature CPU	vccencmd measure_temp
Adresse IP	hostname -I
Redemarrer	sudo reboot
Eteindre	sudo shutdown -h now
Detecter I2C	sudo i2cdetect -y 1

GPIO Python - Resume

Action	Code
Import	import RPi.GPIO as GPIO
Mode BCM	GPIO.setmode(GPIO.BCM)
Sortie	GPIO.setup(pin, GPIO.OUT)
Entree pull-up	GPIO.setup(pin, GPIO.IN, pull_up_down=GPIO.PUD_UP)
Ecrire HIGH	GPIO.output(pin, GPIO.HIGH)
Lire	etat = GPIO.input(pin)
PWM	pwm = GPIO.PWM(pin, freq)
Nettoyage	GPIO.cleanup()

FIN DU COURS RASPBERRY PI 4

Cours + Exemples + Travaux Pratiques
 BTS Electronique - Lycee Moulay Youssef - Tanger
 2025-2026